SGBD P1.1

PRÁCTICA: HUGO + SQLITE, UN PROTOTIPO DIDÁCTICO

Sistemas Gestores de Bases de Datos en Tiempo de Compilación

Alumno: Víctor Verjano
Curso: 2º ASIR
Módulo: SGBD
Profesor: Jordi López Amat (Librebits / fp-bits)
Fecha: Noviembre 2026
Repositorio: victorverjanozaragoza-ops/hugo-sqlite-proto

1. Concepto y Arquitectura

El objetivo fundamental de esta práctica consiste en incorporar un motor de bases de datos relacional (**SQLite**) dentro del pipeline de construcción (*build-time*) de un generador de sitios web estáticos (**Hugo**).

A diferencia de los gestores de contenido dinámicos (CMS como WordPress) donde el servidor de base de datos corre permanentemente en producción atendiendo consultas por cada petición HTTP, en este prototipo la base de datos es puramente **local y transitoria**:

- Se procesan los ficheros Markdown de contenido mediante funciones puras en Python (`tools/index.py`).
- Se inserta la información normalizada en una base de datos local SQLite (`build/site.db`).
- Mediante consultas SQL avanzadas y la extensión **JSON1** de SQLite, se generan directamente los ficheros JSON estáticos (`search.json`, `tags.json`, `stats.json`).
- Hugo consume estos datos para construir el directorio `public/` final. **En producción no existe servidor de base de datos, solo ficheros estáticos HTML, CSS y JS de alto rendimiento.**



2. Resolución de Ejercicios Técnicos

Ejercicio 1: Nuevo post con etiqueta 'sql' y recuento en home

Se ha creado el post `content/posts/sql-en-build-time.md` con la etiqueta `sql` en su front matter YAML. Al ejecutar el proceso de indexación, la consulta `Q_TAGS` realiza un `GROUP BY tag` y `COUNT(*)` automático, reflejándose de forma dinámica en la página principal:

```
---
title: "SQL en tiempo de build con SQLite"
date: 2026-11-10
description: "Cómo indexar y agregar metadatos de un sitio estático usando consultas SQL durante la co
tags: [sql, sgbd, hugo]
---
```

Ejercicio 2: Soporte para flag '--drafts'

Por defecto, `hugo server -D` muestra los borradores (`draft: true`), pero el buscador no los indexaba porque `tools/index.py` los descartaba incondicionalmente. Se ha añadido soporte al script para aceptar el argumento `--drafts` y se ha implementado la regla `make serve-drafts` en el `Makefile`:

```
def read_all(include_drafts=False):
    files = (p for p in sorted(CONTENT.rglob("*.md")) if p.name != "_index.md")
    return [page for page in map(read_page, files) if include_drafts or not page["draft"]]
```

Ejercicio 3: Tabla relacional 'words(url, n)' y métricas en pie de página

Se ha ampliado el esquema de la base de datos añadiendo la tabla `words` con una clave foránea referenciando a `posts(url)`. Una función pura calcula el número de palabras por artículo y una consulta SQL agregada exporta los totales a `data/stats.json`, los cuales son mostrados automáticamente en `layouts/partials/footer.html`:

```
CREATE TABLE words (
  url TEXT PRIMARY KEY REFERENCES posts(url),
  n   INTEGER NOT NULL
);

-- Consulta agregada SQL
SELECT json_object(
  'total_words', COALESCE(SUM(n), 0),
  'avg_words', COALESCE(ROUND(AVG(n), 1), 0),
  'total_posts', COUNT(*)
) FROM words;
```

Ejercicio 4: Búsqueda ordenada por relevancia (función pura)

En `static/search.js` se ha implementado un algoritmo de ponderación funcional sin dependencias. Los resultados de búsqueda ahora se ordenan de mayor a menor relevancia dando mayor peso a las

coincidencias encontradas en el título (x3) y etiquetas (x2) frente al cuerpo del texto (x1).

3. Verificación y Calificación

Comprobación	Comando	Resultado
Validación de JSON generados	`make check`	✅ `check: OK` (valida search.json, tags.json y stats.json)
Compilación estática completa	`make build`	✅ Generación en `public/` sin errores
Servidor con borradores	`make serve-drafts`	✅ Indexa borradores y levanta Hugo en `:1313`
Entrega y Colaboración	Invitación GitHub	✅ Invitación enviada al usuario `fenix011`